Let the Shape of the Data Choose the Model Class

The "Mystery Data" set spans x values from -10 to 10, with y values ranging from roughly -100 up to 350. The structure is unmistakable: at the left edge ($x = -10$), y sits near 300; at the right edge ($x \approx 9$–10), it climbs back to about 280 and 230; but in the middle, around $x = 0$ to 1, the points plunge toward zero, with one observation down near -70. This is neither a rising nor a falling trend — it is a **bowl**.

Recognizing this shape before fitting anything matters because regression models the conditional mean $E(y \mid x)$, and a simple linear model encodes a rigid assumption: every unit increase in x shifts the expected response by the *same* amount, β_1. The bowl violates that assumption outright — moving rightward from $x = -10$ *lowers* y, while moving rightward from $x = 0$ *raises* it. Wikipedia's chemical-synthesis example captures exactly this situation: yield may decrease with increasing temperature over one range and increase over another, so no single slope can describe both regimes. The U-shape tells you in advance that the model class must permit curvature.

The Fitting Machinery: `polyfit`, `polyval`, and Least Squares

The workflow uses two functions in tandem:

```
model1 = pylab.polyfit(xVals, yVals, 1)
pylab.plot(xVals, pylab.polyval(model1, xVals), 'r--', label='Linear Model')
```

- **`pylab.polyfit(xVals, yVals, n)`** finds the parameters of the best-fitting polynomial of order n. With $n = 1$, it returns the values of a and b such that $y = ax + b$ best matches the observed `yVals`.
- **`pylab.polyval(model, xVals)`** evaluates the fitted polynomial at the x values, generating the predicted y's that let you draw the fitted curve through the data.

What "best" means here is the **least squares** criterion: define each residual as the gap between observation and prediction,

$$r_i = y_i - f(x_i, \beta),$$

and choose the parameters minimizing the sum of squared residuals,

$$S = \sum_{i=1}^{n} r_i^2.$$

Because the models being fit are linear in their unknown parameters, this problem has a **closed-form solution**; only genuinely nonlinear models require iterative refinement. The method has deep roots: Legendre published the first clear exposition in 1805, demonstrating it on the same Earth-shape data Laplace had analyzed, and within a decade it became a standard tool in astronomy and geodesy across France, Italy, and Prussia. Gauss published his version in 1809, claiming possession since 1795 (sparking a priority dispute), but went further by connecting least squares to probability theory — in the process inventing the **normal distribution**. The method's power was dramatized when only Gauss's least-squares orbit calculations allowed von Zach to relocate the asteroid Ceres in 1801 after Piazzi lost it in the Sun's glare. Laplace added a large-sample justification via the central limit theorem in 1810, and in 1822 Gauss established optimality: under zero-mean, uncorrelated, equal-variance errors, least squares is the best linear unbiased estimator — the result generalized as the **Gauss–Markov theorem**, which guarantees the least-squares method minimizes the variance of the unbiased coefficient estimators. Notably, when observations come from an exponential family with identity sufficient statistics (normal, exponential, Poisson, binomial), standardized least-squares estimates coincide with maximum-likelihood estimates.

Degree One: A Flat Line That Misses Everything

Fitting with argument $n = 1$ produces a green line that is **essentially flat**, sitting at roughly 100 across the entire range from $x = -10$ to $x = 10$. It never descends to meet the points in the middle and never rises at

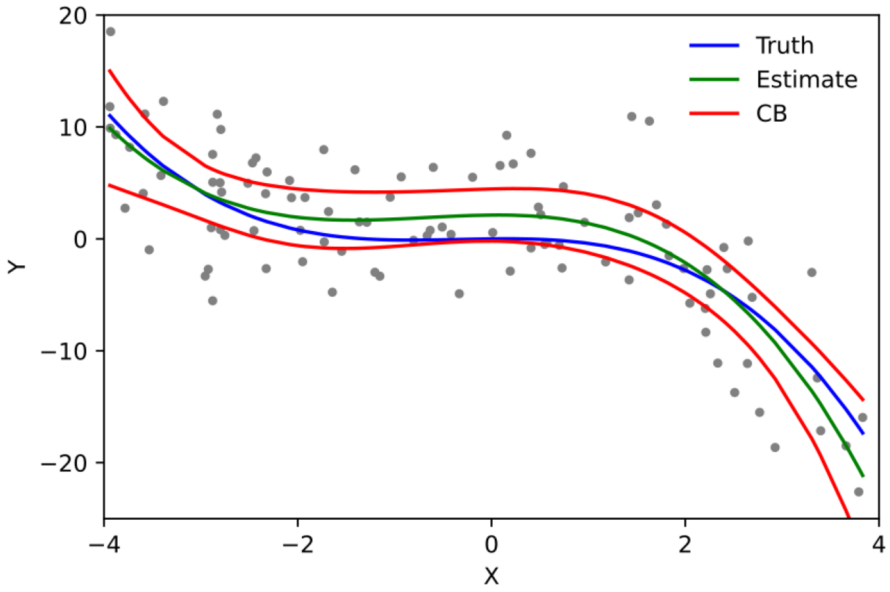
the edges where the data climbs toward 300.

The failure is structural, not numerical. A line possesses a single slope a, but this data demands a negative slope on the left half and a positive slope on the right half. Least squares resolves the conflict by averaging the competing demands into a slope near zero, leaving the intercept near the overall level — about 100. This is close to the degenerate extreme noted in the least-squares theory: if the model is just $f(x, \beta) = \beta$, least squares returns the **arithmetic mean** of the data, i.e., a horizontal line. Here the upward and downward tendencies nearly cancel, so the degree-one fit collapses toward exactly that. A straight line, plainly, is not the right story for this data.

Degree Two: The Quadratic Tracks the Bowl

The upgrade requires changing exactly one character: `model2 = pylab.polyfit(xVals, yVals, 2)`. The final argument of 2 instructs `polyfit` to find the best **quadratic** rather than the best line; plotting proceeds identically via `pylab.polyval`, now labeled `'Quadratic Model'`.

The result is decisive. The red dashed curve starts high on the left, sweeps down through the middle of the data right around zero, and climbs back up on the right — tracking the bowl identified at the outset, while the green line cruises along flat, ignoring everything the data says. The verdict: the quadratic appears to be a better fit.



Source: Wikipedia, article "Polynomial regression".

Why a quadratic succeeds where a line fails comes down to how its effect changes with position. The quadratic model is

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \varepsilon,$$

and increasing x by one unit now changes the expected response by $\beta_1 + \beta_2(2x+1)$; for infinitesimal changes, the total derivative is $\beta_1 + 2\beta_2 x$. The effect of x therefore **depends on x itself** — negative over one range, positive over another — which is precisely what a bowl requires and what a constant slope cannot express. This dependence of the change on x is what makes the relationship nonlinear, and the same logic extends to modeling $E(y \mid x)$ as an nth-degree polynomial in general.

Nonlinear Curve, Linear Estimation

Here lies the key subtlety that makes tools like `polyfit` work so cleanly. Although polynomial regression fits a **nonlinear relationship** between x and $E(y \mid x)$, as a statistical estimation problem it is **linear**: the regression function is linear in the unknown parameters $\beta_0, \beta_1, \ldots$. Polynomial regression is therefore a special case of **multiple linear regression** — one simply treats $x, x^2, \ldots, x^m$ as distinct independent variables. The explanatory variables produced by the polynomial expansion of the baseline variable are known as **higher-degree terms**, and such variables also appear in classification settings.

In matrix form, the model is written using a **design matrix X** (whose i-th row contains the powers of x_i), a response vector $\vec{y}$, a parameter vector $\vec{\beta}$, and an error vector $\vec{\varepsilon}$. Ordinary least squares yields the estimated coefficients

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \vec{y},$$

which requires $m < n$ for the matrix to be invertible. Because $\mathbf{X}$ is a **Vandermonde matrix**, that invertibility is guaranteed whenever all the x_i values are distinct. This is the unique least-squares solution: it minimizes the distance $\|\vec{\varepsilon}\|$ between the observed y_i and the corresponding polynomial values $\sum_{k=1}^{m} \beta_k x_i^k$. For direct implementation, the normal equations expand into a system built entirely from power sums $\sum_i x_i^k$ arranged in a symmetric matrix, matched against the mixed sums $\sum_i y_i x_i^k$ on the right-hand side. The practical payoff: all the computational and inferential machinery of multiple regression transfers wholesale to polynomial fitting — which is why `polyfit` solves the problem in closed form rather than by iteration.

Better Fit Is Not Yet the Right Model

The lesson of the exercise: when a linear model clearly fails to capture the structure of your data, moving to a higher-degree polynomial can produce a dramatically better fit. But hold that thought — **"better fit" and "right model" are not always the same thing**, and there is a danger in simply cranking up the degree. That tension is where the story goes next.

History offers a cautionary footnote in the same spirit. Polynomial regression played a central role in the development of regression analysis throughout the twentieth century, with emphasis on design and inference — yet more recently, polynomial models have been complemented by other methods, with non-polynomial approaches holding advantages for some classes of problems. Degree is a powerful dial, but not the only one, and turning it blindly is not automatically progress.

Cross-validation: train on one dataset, test on another

The slide sets up a deliberate mismatch. Having fit a family of polynomial models to each dataset separately (`models1` from dataset 1, `models2` from dataset 2), the test code opens a figure and calls `testFits(models1, degrees, xVals2, yVals2, 'DataSet 2/Model 1')` — the models *trained* on dataset 1, but *evaluated* on the x and y values of dataset 2 — and then, symmetrically, `testFits(models2, degrees, xVals1, yVals1, 'DataSet 1/Model 2')`. Every model is scored on data it never saw during fitting. The plot titles encode exactly this: "DataSet 2/Model 1" means data from set 2, models from set 1.

This procedure is **cross-validation**, also called *rotation estimation* or *out-of-sample testing*:

Why Extra Complexity Can Never Hurt the Training Fit

The central puzzle of this part of the lecture: a higher-order model gives a *better* fit on the training data, yet does *worse* on new data. What actually happens when we increase the polynomial order during training?

Start with the question that must be answered first: **can adding a term ever make the fit to the training data worse?** The answer is no. If the extra term is useless, its coefficient will simply come out as zero. The reason is structural: a degree-$k+1$ polynomial contains every degree-k polynomial as a special case (just set the leading coefficient to zero). Least squares chooses the coefficients minimizing the residual

sum of squares over the whole family, so the best fit in the larger family can only match or beat the best fit in the smaller one. As the Wikipedia article on polynomial regression explains, such models express the conditional mean $E(y \mid x)$ as a polynomial in x, and although the relationship is nonlinear in x, it is *linear in the unknown parameters* $\beta_0, \beta_1, \ldots$ — so fitting reduces to ordinary least squares, with $x, x^2, \ldots$ treated as distinct regressors in a multiple regression.

But here is the catch the professor flags immediately: **if the data is noisy, the model can fit the noise rather than the underlying pattern**, yielding a "better" R^2 without being a genuinely better fit. Those scare quotes do a lot of work. The Wikipedia article on overfitting names this precisely: the essence of overfitting is *unknowingly extracting some of the residual variation (i.e., noise) as if that variation represents the underlying model structure.* Two concrete experiments make this vivid.

Baseline Experiment: A Quadratic Fit to Perfect Data Costs Nothing

First, deliberately over-complicate a dataset that needs no complication. Set `xVals = (0, 1, 2, 3)` and `yVals = xVals`, so the true relationship is exactly $y = x$. Instead of fitting a line, fit a quadratic with `pylab.polyfit(xVals, yVals, 2)`, obtaining coefficients (a, b, c) for the model

$$y = ax^2 + bx + c,$$

then evaluate it at the x values with `polyval` and compute R^2.

The result: $a = 0$, $b = 1$, $c = 0$ — the fit is just $y = x$. The quadratic coefficient came out **exactly zero**, exactly as promised: the extra term was useless, so `polyfit` didn't use it. And $R^2 = 1.0$, a perfect fit, with the predicted curve lying right on top of the actual data. Adding complexity to a clean dataset cost nothing.

Now test the same model on new data. Extend `xVals` by appending `(20,)` — note the syntax: the trailing comma is what makes a single-element tuple — set `yVals = xVals` again, and evaluate the *same* coefficients at these points. Since the model is just $y = x$, it predicts $x = 20$ perfectly: R^2 remains 1.0 and the two curves coincide all the way out to 20. On noiseless data, the extra parameter had no residual variation to absorb, so training performance and generalization happened to agree.

The Key Experiment: One Small Measurement Error Gets Absorbed into Curvature

Real experimental data is never that clean, so simulate a small measurement error: same `xVals = (0, 1, 2, 3)`, but now `yVals = (0, 1, 2, 3.1)` — the last point is off by just a tenth. Fit a degree-2 polynomial again, print the model, evaluate, plot, and compute R^2.

The fitted model is now

$$y = 0.025x^2 + 0.955x + 0.005.$$

The quadratic coefficient is no longer zero. That tiny measurement error got absorbed into the curvature of the fit. Mechanically, least squares trades a small amount of spurious curvature for a reduction in squared error at the perturbed point — with four observations and three free parameters, the fit has enough freedom to do this. This is the Wikipedia article's definition of an overfitted model in action: *a model that contains more parameters than can be justified by the data*, where for polynomials those parameters represent the degree. The extreme version, per the same article, is when the number of parameters equals or exceeds the number of observations — then the model can perfectly predict the training data simply by memorizing it, and will typically fail severely at prediction. Four points against three parameters is uncomfortably close to that regime.

And the diagnostic that should worry you: $R^2 = 0.9994$. If all you did was look at R^2, you would call this a nearly perfect model. On the plotted region between 0 and 3, the predicted curve is almost indistinguishable from the data.

Extrapolation: Where the Tiny Quadratic Term Takes Over

Do exactly what worked before: append `(20,)` to `xVals`, set `yVals = xVals`, evaluate the same model, and print R^2.

The model fails badly. The actual value at $x = 20$ is 20, but the prediction shoots up to almost 29, and R^2 collapses to 0.7026.

The mechanism is pure arithmetic on the fitted coefficients. Near the training data ($0 \leq x \leq 3$), the term $0.025x^2$ is negligible — at $x = 3$ it contributes only $0.025 \cdot 9 = 0.225$ — which is why the fit looked so good and R^2 was so high. But a quadratic term grows without bound relative to the linear one. At $x = 20$:

$$0.025(20)^2 + 0.955(20) + 0.005 = 10 + 19.1 + 0.005 \approx 29.1,$$

so the curvature term now contributes roughly a third of the prediction and the model overshoots by about 9 units. A measurement error of 0.1 at a *single* point produced a small but systematically wrong quadratic component that dominates entirely once you leave the training range.

This is exactly the phenomenon flagged earlier: the noisy data let the model fit the noise rather than the underlying pattern. We got a "better" R^2 on the training data, but not a better fit — and on new data the model failed us.

Overfitting: The General Lesson

This example is the canonical warning about overfitting, and the Wikipedia material generalizes it. **Overfitting** is the production of an analysis that corresponds too closely or exactly to a particular set of data and may therefore fail to fit additional data or predict future observations reliably. Its mirror image, **underfitting**, occurs when a model cannot adequately capture the underlying structure — for instance, fitting a linear model to nonlinear data — and also yields poor predictive performance. A best approximating model properly *balances* the errors of underfitting and overfitting (Burnham & Anderson's argument for the Principle of Parsimony).

Two structural points explain why this trap is so easy to fall into:

- **Mismatched criteria.** The possibility of overfitting exists whenever the criterion used to *select* the model differs from the criterion used to *judge* it — e.g., maximizing performance on training data while suitability means performing well on unseen data. Overfitting occurs when a model begins to "memorize" training data rather than "learning" to generalize from a trend.
- **Shrinkage is normal; collapse is not.** Even a correctly sized model should be expected to perform somewhat less well on new data than on the fitting data — the coefficient of determination shrinks relative to the original data. A modest drop is expected; a plunge like $0.9994 \to 0.7026$ signals that the model captured noise.

The statistical consequences of keeping unjustified parameters are concrete: overfitted models may have unbiased parameter estimates but *needlessly large sampling variances* (poor precision relative to a more parsimonious model), and they tend to identify false treatment effects and include false variables. Related pathologies include **Freedman's paradox** — with many explanatory variables that have no real relation to the outcome, some will falsely appear statistically significant and get retained — and the **bias–variance tradeoff**, which decomposes a regression function's mean squared error into random noise, approximation bias, and variance in the estimate, and is the standard lens for taming overfit models.

Finally, the remedies, each attacking one side of the problem: penalize complexity explicitly (**regularization**, Bayesian priors) or test generalization directly on data withheld from training (**cross-validation**, model comparison), plus techniques like early stopping, pruning, and dropout.

The takeaway to carry forward: **a high R^2 on your training data is not, by itself, evidence that your model is any good.** Recall what R^2 measures,

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}},$$

the proportion of variation in the outcomes explained by the model. A sufficiently flexible model can drive SS_{res} toward zero on the training set whether it has captured the underlying structure or merely the noise — and only evaluation on data the model never saw can tell the difference.

Overfitting versus underfitting: the case for the simpler model

The lecture opens this segment with a counterfactual: take the dataset previously fit with a second-degree polynomial and refit it with `pylab.polyfit(xVals, yVals, 1)` — the only change being the degree argument, here 1. The result is striking: the blue solid line of actual values and the red dashed line of predicted values lie almost exactly on top of one another from zero out to twenty, with $R^2 = 0.9988$ — essentially a perfect fit. Placed side by side with the degree-2 model, the difference is immediate. With the degree-2 polynomial, the predicted values peel away from the actual values during extrapolation: by $x = 20$ the prediction reaches roughly 29 while the actual value is only about 20. The model is "extrapolating nonsense." With the degree-1 polynomial, predicted and actual values are virtually indistinguishable across the whole range. The predictive ability of the first-order fit is far better — and what we care about is precisely *not* how well the model reproduces its training data, but how it performs on data it has never seen.

This is the textbook definition of **overfitting**: the production of an analysis that corresponds too closely or exactly to a particular set of data and therefore fails to fit additional data or predict future observations reliably. An overfitted model contains more parameters than can be justified by the data — and for a polynomial model, those parameters *are* the degree. The essence of overfitting is unknowingly extracting some of the residual variation (i.e., noise) as if it represented the underlying structure. The mechanism is a mismatch of criteria: the model is *selected* by maximizing performance on training data, yet its *suitability* is judged by performance on unseen data. Overfitting occurs when the model begins to "memorize" the training data rather than learn to generalize from a trend. The extreme case makes this vivid: if the number of parameters equals or exceeds the number of observations, the model can perfectly predict the training data simply by memorizing it — in linear regression, p variables with p data points put the fitted line exactly through every point — and such a model typically fails severely when making predictions.

The opposite failure is **underfitting**: a model that cannot adequately capture the underlying structure of the data because it is missing parameters or terms a correctly specified model would contain. Fitting a linear model to nonlinear data is the canonical example — exactly what happened earlier in the lecture when a straight line was fit to fundamentally parabolic data and simply could not capture the curvature. Such models tend to have poor predictive performance.

Hence the take-home message: choosing an overly complex model leads to overfitting and increases the risk of poor performance on data outside the training set, while choosing an insufficiently complex model misses real structure. Einstein's dictum captures the target: "Everything should be made as simple as possible, but not simpler" — simple enough to generalize, complex enough to capture the physics. Statistical theory reinforces this balance. Burnham & Anderson's Principle of Parsimony notes that overfitted models, while often free of bias in their parameter estimators, carry needlessly large sampling variances (poor precision relative to a more parsimonious model) and tend to identify false treatment effects and include false variables; the best approximating model properly balances the errors of underfitting against those of overfitting. Even a correctly sized model exhibits *shrinkage*: its coefficient of determination will shrink somewhat on new data relative to the fitting data, so some degradation is normal — catastrophic degradation, like the degree-2 extrapolation above, is the signature of genuine overfitting.

Searching for model complexity when theory is silent

If no theory predicts the order of the model, the practical remedy is a **search process**:

1. Fit a low-order model to the training data.
2. Test it on new data and record the R^2 value.

3. Increase the order of the model and repeat.
4. Stop when the fit on the *test* data begins to decline — you have found your complexity.

Two datasets show both outcomes of this search. On **DataSet 1**, test R^2 climbs steadily: 0.86088 at degree 2, 0.87628 at degree 4, 0.89929 at degree 8, and 0.99615 at degree 16, where the magenta fitted curve snakes through almost every point — adding terms keeps genuinely helping. On **DataSet 2**, the pattern reverses: 0.86721 at degree 2 and 0.86917 at degree 4 (a trivial improvement), then a drop to 0.83409 at degree 8 and a collapse to 0.69967 at degree 16. That wiggly degree-16 curve is chasing the noise in the training data and does *worse* on the test data than the humble parabola did. The search process tells you to stop early — that is what balancing fit against complexity looks like in practice.

The Wikipedia account explains why this situation arises and why the search works. Overfitting is a particularly serious concern when little theory is available to guide the analysis, because then there tend to be a large number of candidate models to select from — "given a data set, you can fit thousands of models at the push of a button, but how do you choose the best?" A function class that is too large relative to the dataset size is likely to overfit. Countermeasures fall into two families: (1) explicitly penalize overly complex models (regularization, pruning, Bayesian priors, early stopping, dropout, model comparison), or (2) test the model's ability to generalize by evaluating it on data not used for training — which is exactly what the search recipe does, and what cross-validation formalizes. In regression terms, the mean squared error of the fitted function decomposes into random noise, approximation bias, and variance in the estimate of the regression function; the bias–variance tradeoff is the lens for choosing among models. A related caution is Freedman's paradox: with a large set of explanatory variables that actually have no relation to the dependent variable, some will falsely appear statistically significant and be retained, silently overfitting the model — another reason increasing complexity must be checked against held-out data, not training fit alone.

When theory speaks: Hooke's law and the elastic limit

Returning to the spring experiment — measured displacement in meters plotted against the magnitude of the applied force in Newtons — the statistics tempt us astray. The linear fit gives $r^2 = 0.88151$; the quadratic fit gives $r^2 = 0.95416$. Statistically, the quadratic is tighter; it hugs the points better. But remember Hooke. **Hooke's law** is the empirical law stating that the force needed to extend or compress a spring by some distance scales linearly with that distance:

$$F_s = kx, \qquad x = \frac{F_s}{k},$$

where k is a positive constant characteristic of the spring (its stiffness) and x is small compared to the total possible deformation. Under the restoring-force convention the equation reads $F_s = -kx$, the minus sign indicating that the force opposes the displacement. Robert Hooke first stated the law in 1676 as a Latin anagram and published its solution in 1678: *ut tensio, sic vis* — "as the extension, so the force" — though he reported being aware of it since 1660. Geometrically, the graph of applied force versus displacement is a straight line through the origin whose slope is k. The law is the fundamental principle behind the spring scale, the manometer, the galvanometer, and the balance wheel of the mechanical clock.

Crucially, Hooke's law is a *first-order linear approximation* to the real response of springs and elastic bodies. It

Splitting the data randomly: the `splitData` function

Holding data back starts with a concrete mechanism. The function `splitData(xVals, yVals)` does the partitioning, and its first line carries all the weight:

```
toTrain = random.sample(range(len(xVals)), len(xVals)//2)
```

This draws `len(xVals)//2` indices — exactly half the data points — uniformly at random **without replacement** from the full range of indices. Because `random.sample` never picks the same index twice, `toTrain` is a genuine random half of the dataset, not a half with duplicates. The rest of the function is bookkeeping:

initialize four empty lists (`trainX`, `trainY`, `testX`, `testY`), loop over every index `i`, append the point to the training lists if `i` is in `toTrain` and to the test lists otherwise, then return all four lists.

The crucial design decision is that the split is *random*. We do not assign the first half of the data to training and the second half to testing — we are effectively flipping coins for each point. If the data have any ordering (by time, by magnitude, by collection batch), a sequential split would make the two sets systematically different; a random split makes both sets representative of the same underlying population. This is exactly the operation cross-validation formalizes: one round consists of **partitioning a sample into complementary subsets**, performing the analysis on one subset (the *training set*) and validating it on the other (the *validation* or *testing set*). A 50/50 random split is the simplest instance of that idea.

Train, test, report: fitting on one subset, scoring on the other

With `splitData` in hand, the experimental loop on slide 47 runs the full protocol once per trial:

```
for f in range(numSubsets):          # fresh random split each trial
    trainX, trainY, testX, testY = splitData(xVals, yVals)
    for d in dimensions:             # candidate model complexities
        model = pylab.polyfit(trainX, trainY, d)   # fit ONLY on training data
        #estYVals = pylab.polyval(model, trainX)   # deliberately NOT this
        estYVals = pylab.polyval(model, testX)     # apply to held-out data
        rSquares[d].append(rSquared(testY, estYVals))
```

Two details deserve emphasis. First, the commented-out line is there on purpose: we deliberately do **not** evaluate the model on the data it was fit to. Second, the active line applies the fitted polynomial to `testX` and scores those predictions against `testY`. After all trials finish, the summary prints, for each dimensionality, the mean of the collected R-squared values and their standard deviation (`numpy.std`), each rounded to four decimal places.

Why insist on scoring out-of-sample? The statistics are unambiguous about what happens if you don't. Cross-validation exists precisely to "test the model's ability to predict new data that was not used in estimating it," flagging problems like overfitting and selection bias. When validation data are drawn from the same population as the training data, a fitted model almost always fits the validation data *worse* than the training data — and the gap grows when the training set is small or the model has many parameters. In linear regression fit by least squares, this gap can even be quantified: if the model is correctly specified, the expected training-set MSE is only

$$\frac{n-p-1}{n+p+1} < 1$$

times the expected validation-set MSE, where n is the number of observations and p the number of parameters. Training-set performance is therefore an *optimistically biased*, in-sample estimate; the held-out score is an *out-of-sample* estimate. The overfitting literature frames the same hazard as a mismatch of criteria: a model selected by maximizing performance on training data may simply be "memorizing" the training data rather than learning to generalize — and even an honestly fit model exhibits *shrinkage*, performing somewhat less well on new data, with the coefficient of determination shrinking relative to its value on the original data.

The quantity being accumulated, R-squared, is the coefficient of determination:

$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

the proportion of the variation in the dependent variable that is predictable from the independent variable(s). Perfect predictions give $SS_{\text{res}} = 0$ and $R^2 = 1$; a baseline that always predicts the mean $\bar{y}$ gives $R^2 = 0$. One subtlety matters here: negative values of R^2 arise when the predictions were *not* derived from a model-fitting procedure on the very data being scored — which is exactly our situation, since the model was fit on the

training half and scored on the test half. A sufficiently bad model can therefore score below zero on held-out data, meaning the test-set mean would have been a better predictor than the model.

Finally, note the outer structure: `numSubsets` repetitions, each with a *different* random partition, whose results are then averaged. This mirrors the standard practice in cross-validation of performing multiple rounds over different partitions and combining (e.g., averaging) the validation results, because a single round is noisy — a point the next section makes vividly.

The verdict: the straight line wins on every count

Running the loop over dimensionalities $d = 1, \ldots, 4$ produced:

Dimensionality	Mean R^2	Std
1	0.7535	0.0656
2	0.7291	0.0744
3	0.7039	0.0684
4	0.7169	0.0777

The degree-1 fit is the winner on **three independent grounds**: it has the highest average test R-squared, the smallest deviation across trials, and it is the simplest model. Adding complexity never helped — the cubic ($d = 3$) actually posted the *lowest* mean. This is the signature of mild overfitting in action: the essence of overfitting is unknowingly extracting some of the residual variation (noise) as if it were underlying structure, and extra polynomial terms buy flexibility that fits noise rather than signal. Model-selection theory says to expect exactly this. Burnham & Anderson's Principle of Parsimony holds that overfitted models, though often unbiased, carry "needlessly large" sampling variances — poorer precision than a more parsimonious model would achieve — and indeed the higher-degree fits show larger standard deviations here. Overfitting is likewise a violation of Occam's razor: including more adjustable parameters than are ultimately optimal.

Notice also what the deviations tell us: they are roughly an order of magnitude smaller than the means (e.g., 0.0656 against 0.7535), indicating good agreement across trials — but they are still large enough that individual trials vary noticeably. That observation motivates the final question.

Why run many trials: one unlucky split can mislead

Suppose we had been lazy and run the experiment exactly once. Here are the actual R-squared values from the ten trials of the linear fit:

0.7828, 0.8064, 0.7964, 0.7843, 0.7600, **0.5709**, 0.7212, 0.7436, 0.7903, 0.7792

Nine of the ten cluster between roughly 0.72 and 0.81 — consistent with the mean of 0.7535. But one unlucky random split produced an R-squared of only about **0.57**. Had that been our single trial, we would likely have concluded that a line fits poorly and reached for a more complex model. That conclusion would have been wrong: we would have been chasing noise, not signal — reacting to which particular points happened to land in the test set rather than to any property of the underlying process. This is the statistical argument for repetition baked into cross-validation: "to reduce variability, in most methods multiple rounds of cross-validation are performed using different partitions, and the validation results are combined (e.g., averaged)," yielding a more accurate estimate of predictive performance than any single round. It also echoes a broader warning from the model-selection literature: with noisy evidence, false effects and spurious variables tend to be identified and retained — a trap that a single misleading split invites.

Takeaways: models, prediction, and how to choose complexity

The curve-fitting exercise distills into four durable lessons:

1. **Regression is a mapping.** Linear regression fits a curve to data — a mapping from independent variable values to dependent variable values.
2. **The curve is a model for prediction.** We use it to predict the dependent values associated with independent values we have not seen — *out-of-sample* data. That is the entire point of building the model.
3. **R-squared evaluates, but higher is not automatically better.** R^2 measures the fraction of variation explained, but a more complex model can look excellent on the data it was trained on while predicting worse. An overfitted model contains more parameters than the data justify; in the extreme, a linear regression with p parameters fit to p data points passes exactly through every point by pure memorization and then "fails severely" at prediction. Test-set evaluation is the antidote.
4. **Choosing complexity rests on three pillars:** *theory* about the structure of the underlying process (what do we believe generated the data?), *cross-validation* — the train/test procedure just implemented — and *simplicity*: all else being equal, prefer the simpler model.

These three criteria are not specific to polynomials; they govern model choice wherever flexible models meet finite, noisy data, and they will recur well beyond curve fitting.

Computational Thinking: Formulating Problems So Computers Can Solve Them

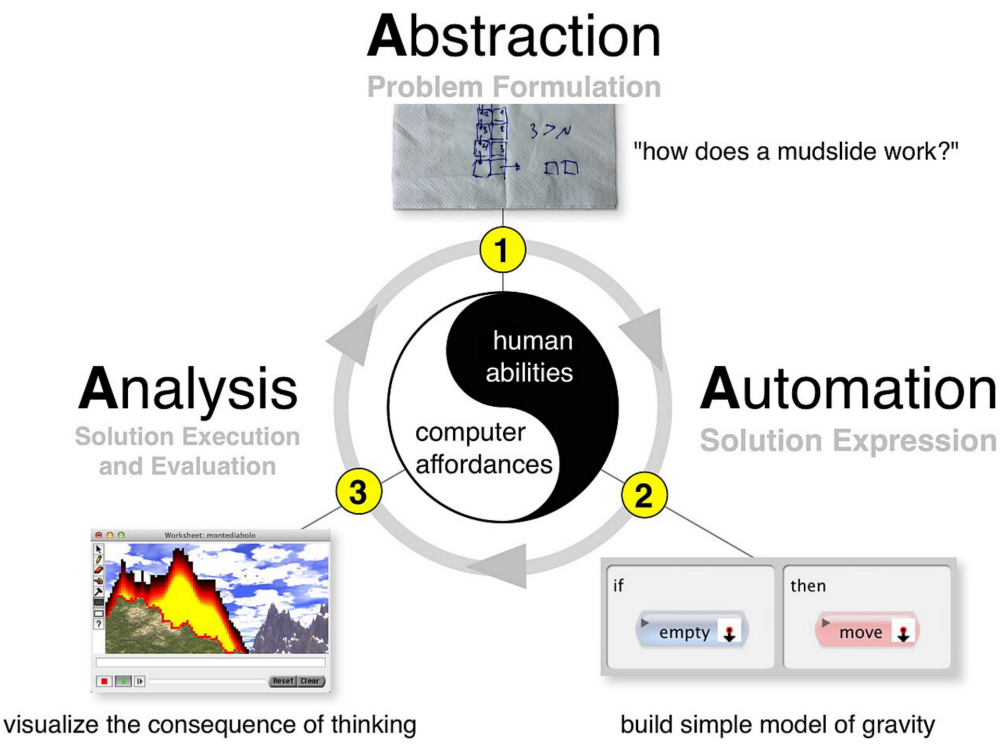
Computational thinking (CT) is the set of thought processes involved in formulating problems so that their solutions can be represented as computational steps and algorithms. In educational terms, it means expressing problems *and* their solutions in ways a computer could also execute. Crucially, it works in two directions: it automates processes, but it also uses computing to explore, analyze, and understand processes — both natural and artificial. This dual role is why CT matters beyond programming: it is a lens for understanding the world, not just a tool for building software.

The idea is far older than its current popularity. Its history as a concept dates back at least to the 1950s, and most of its constituent ideas are older still — earlier vocabulary included "algorithmizing," "procedural thinking," "algorithmic thinking," and "computational literacy," championed by computing pioneers such as Alan Perlis and Donald Knuth. Seymour Papert first used the exact term "computational thinking" in 1980 and again in 1996. What brought the phrase to the forefront of computer science education was Jeannette Wing's 2006 essay in *Communications of the ACM*. Wing argued that thinking computationally is a **fundamental skill for everyone**, not just computer scientists, and pushed for integrating computational ideas into other school subjects — even suggesting that children who learn CT become better at mundane everyday tasks like packing a backpack, finding lost mittens, or knowing when to stop renting and buy instead. Like Papert, Perlis, and Marvin Minsky before her, she envisioned CT becoming an essential part of every child's education; the "algoRithms" component has even been called the "fourth R," joining reading, wRiting, and aRithmetic.

The field characterizes CT through four defining elements:

- **Decomposition** — breaking a problem into manageable parts;
- **Pattern recognition / data representation** — identifying the variables involved and how to encode them;
- **Generalization / abstraction** — distilling the structure of the problem;
- **Algorithms** — specifying the steps of a solution.

These compose powerfully: by decomposing a problem, identifying its variables through data representation, and building algorithms, you arrive at a *generic* solution — a generalization or abstraction that solves not just the original problem but a multitude of its variations. That is precisely why CT "can be used to algorithmically solve complicated problems of scale" and often yields large efficiency improvements. A complementary characterization is the iterative **"three As"** process — abstraction, automation, analysis — which frames CT as a repeatable cycle rather than a one-shot procedure:



Source: Wikipedia, article "Computational thinking".

CT shares DNA with neighboring modes of thought — scientific, engineering, systems, design, and model-based thinking all involve abstraction, data representation, and logical organization of data. That overlap is a strength (it transfers across subjects, including social sciences and language arts, not just STEM) but also a documented challenge: researchers still struggle to agree on a single definition of CT, to assess children's development in it, and to distinguish it cleanly from those similar "thinkings." Some have even proposed CT as a "fifth C" alongside the four Cs of 21st-century learning (communication, critical thinking, collaboration, creativity).

Adoption has been global and rapid. The United Kingdom has had CT in its national curriculum since 2012; Singapore calls it a "national capability"; Australia, China, Korea, and New Zealand have mounted large-scale efforts to introduce it in schools; and in the United States, President Obama launched the "Computer Science for All" program to equip a new generation with the proficiency needed in a digital economy. One caveat worth remembering: for its first decade CT was a largely US-centered movement, and the field's most-cited articles, people, and researcher networks remain US-based — raising open questions about whether a predominantly Western literature serves students in other cultural groups. An emerging globalization effort comes from the Prolog community, whose Prolog Education Committee (sponsored by the Association for Logic Programming) aims to make computational and logical thinking through Prolog and its successors a core subject worldwide.

Data Science: Extracting Knowledge from Noisy Data

Data science is an interdisciplinary academic field that uses statistics, scientific computing, scientific methods, processing, scientific visualization, algorithms, coding (languages like Python, SQL, and R), and systems to extract or extrapolate knowledge from potentially noisy, structured, or unstructured data. A data scientist, concretely, is a professional who creates programming code and combines it with statistical knowledge to summarize data. The field's practical importance lies in modern decision-making: it enables organizations to extract actionable insights from large and complex datasets, and it integrates domain knowledge from the underlying application area — natural sciences, information technology, medicine — because raw computation

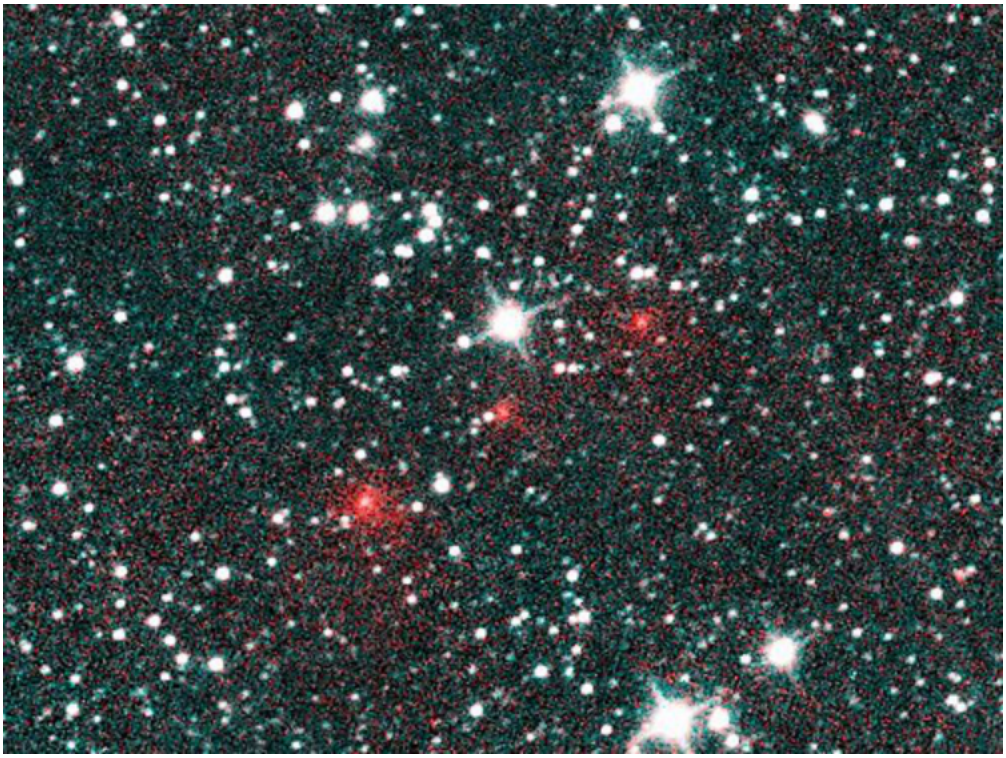
without domain understanding rarely yields meaningful conclusions.

Data science is deliberately hard to pin down: it has been described variously as a science, a research paradigm, a research method, a discipline, a workflow, and a profession. At its core it is "a concept to unify statistics, data analysis, informatics, and their related methods" in order to "understand and analyze actual phenomena" with data, drawing techniques and theory from mathematics, statistics, computer science, information science, and domain knowledge — yet it remains distinct from computer science and from information science. Its scope is broad: preparing data for analysis, formulating data-science problems, analyzing data, and summarizing findings, incorporating skills from computer science, mathematics, data visualization, graphic design, communication, and business.

Its relationship to statistics deserves care, because the boundaries are contested:

- Vasant Dhar draws the line by data type and goal: statistics emphasizes quantitative data and *description*, whereas data science handles quantitative **and** qualitative data (images, text, sensor streams, transactions, customer information) and emphasizes *prediction and action*.
- Andrew Gelman goes further, describing statistics as merely a non-essential *part* of data science.
- David Donoho pushes back on hype from the other side: data science is **not** distinguished from statistics by dataset size or use of computing, and many graduate programs misleadingly advertise analytics and statistics training as the essence of data science. He frames it as an applied field growing out of traditional statistics.

The deepest framing comes from Turing Award winner Jim Gray, who imagined data science as the **"fourth paradigm" of science** — after the empirical, theoretical, and computational paradigms comes the data-driven one — asserting that "everything about science is changing because of the impact of information technology" and the resulting data deluge. A vivid instance of this paradigm in action: the existence of Comet NEOWISE was discovered purely by analyzing astronomical survey data acquired by the Wide-field Infrared Survey Explorer space telescope — the comet appears as nothing more than a series of red dots in processed imagery, yet that signal, mined from noise, was a genuine astronomical discovery:



Source: Wikipedia, article "Data science".

The field's history shows gradual convergence rather than sudden invention. John Tukey described a field he called "data analysis" as early as 1962 that closely resembles modern data science. Peter Naur traced the term itself back to his 1974 book *Concise Survey of Computer Methods*, proposing "data science" as an alternative name for computer science to reflect the growing emphasis on data-driven methods. C. F. Jeff Wu used "data science" as an alternative name for statistics in a 1985 lecture at the Chinese Academy of Sciences in Beijing, and suggested the rename again in 1997 — reasoning that a new name would help statistics shed inaccurate stereotypes, such as being synonymous with accounting or limited to describing data. Milestones followed in quick succession: a 1992 symposium at the University of Montpellier II acknowledged the emergence of a new discipline combining statistics and data analysis with computing for data of various origins and forms; in 1996 the International Federation of Classification Societies became the first conference to feature data science specifically; in 1998 Hayashi Chikio argued for data science as a new interdisciplinary concept with three aspects — data design, collection, and analysis. The modern independent discipline is sometimes attributed to William S. Cleveland. On the professional side, the title "data scientist" is credited to DJ Patil and Jeff Hammerbacher in 2008 (though the National Science Board had used the phrase broadly in its 2005 report on digital data collections), and in 2012 Thomas H. Davenport and DJ Patil declared it "the Sexiest Job of the 21st Century" — a catchphrase picked up even by major newspapers like the *New York Times* and *Boston Globe*, and reaffirmed a decade later with the observation that the job is more in demand than ever. Institutional recognition followed: in 2014 the American Statistical Association's Section on Statistical Learning and Data Mining renamed itself the Section on Statistical Learning and Data Science.

At the working level, data analysis typically means operating on structured datasets to answer specific questions or solve specific problems — tasks such as data cleaning and data visualization to summarize data and develop hypotheses. This is the practical loop where everything above becomes concrete: clean the data, look at it, summarize it, and let the summaries drive the next question.

Where the Two Fields Meet

The pairing of these two subjects is not accidental — they are complementary halves of a single way of working. Computational thinking supplies the *problem-formulation* machinery: decomposing a messy real-world question, representing its variables as data, abstracting away incidental detail, and designing algorithms whose generic solutions handle whole families of problem variants. Data science supplies the *knowledge-extraction* engine: the statistical methods, code, and visualization systems that turn noisy, structured or unstructured data into actionable insight. Each pillar reinforces the other. CT's emphasis on using computing "to explore, analyze, and understand processes" is exactly the stance a data scientist takes toward a dataset; conversely, data science's tools — algorithms, coding, scientific visualization — are the concrete instruments through which the abstractions of computational thinking get executed and tested against reality. And both fields share the same cultural trajectory Wing envisioned: the conviction that these are fundamental skills for everyone, now being written into national curricula and redefining what it means to reason in a data-saturated world.